Interaktív feladatok

Kétszemélyes játékok

Szerző: Nikházy László
Előadó: Leitereg András

2026. július 1.

Horváth Gyula diasorának felhasználásával

A fejezetben olyan kétszemélyes játékokkal foglalkozunk, amelyek **determinisztikusak** (nincs véletlen), **teljes információjúak** (minden állapot ismert) és **végesek**.

A kimenetelük és a célunk alapján a játékokat **két fő típusra** osztjuk:

1. **Bináris kimenetelű (nyerő/vesztő) játékok:** A játék végén nincs pontszám, csak győzelem vagy vereség. A célunk egy olyan **nyerő stratégia** biztosítása, amellyel az ellenfél lépéseitől függetlenül garantált a győzelem.

2. **Nem-bináris (pontgyűjtő) játékok:** A játékosok lépéseikkel pontokat gyűjtenek vagy funkcionális értékeket módosítanak. A célunk a **saját pontszám maximalizálása** (és az ellenfélének minimalizálása) az optimális lépéssorozattal.

Bináris kimenetelű játékok

Nyerő és vesztő pozíciók, pálcikás játékok, Nim és a Sprague–Grundy tétel

Nyerő és vesztő pozíciók

A játékok állapotait (pozícióit) két kategóriára oszthatjuk fel, feltételezve, hogy mindkét fél optimálisan játszik:

- **Nyerő pozíció (NY):** Az az állapot, amelyből a soron következő játékos ki tud kényszeríteni győzelmet.
- **Vesztő pozíció (V):** Az az állapot, amelyből a soron következő játékos mindenképpen veszít, ha az ellenfele nem hibázik.

Az osztályozás alapszabályai:

1. Egy végállapot (ahonnan már nem lehet lépni) **vesztő pozíció**.
2. Ha egy állapotból **van legalább egy lépés** egy vesztő pozícióba, akkor az az állapot **nyerő**.
3. Ha egy állapotból **minden lehetséges lépés** nyerő pozícióba vezet, akkor az az állapot **vesztő**.

Példa – Pálcika elvevő játék

Szabályok: Adott egy `n` pálcikából álló halom. Két játékos (`A` és `B`) felváltva lép, `A` kezd. Egy lépésben 1, 2 vagy 3 pálcikát lehet elvenni. Az nyer, aki az utolsó pálcikát elveszi.

Példa – Pálcika elvevő játék

Szabályok: Adott egy `n` pálcikából álló halom. Két játékos (`A` és `B`) felváltva lép, `A` kezd. Egy lépésben 1, 2 vagy 3 pálcikát lehet elvenni. Az nyer, aki az utolsó pálcikát elveszi.

Pozíciók osztályozása (0 . . . 12 pálcika esetén):

n	0	1	2	3	4	5	6	7	8	9	10	11	12
Áll.	V	NY	NY	NY	V	NY	NY	NY	V	NY	NY	NY	V

Stratégia:

- A 0 pálcika vesztő (`V`), mert nem lehet lépni.
- Az 1, 2, 3 nyerő (`NY`), mert léphetünk 0-ra.
- A 4 vesztő (`V`), mert csak 1, 2, 3-ra léphetünk, amik mind nyerők az ellenfélnek.
- **Szabály:** `n` akkor vesztő, ha 4-gyel osztható. Az optimális stratégia az, hogy a lépésünk után a pálcikák száma mindig 4 többszöröse legyen.

Egy másik pálcikás játék (Osztó-szabály)

Szabályok: Egy halomból k pálcika esetén tetszőleges olyan x számú pálcika elvehető, amelyre igaz, hogy $x < k$ és x osztója k-nak. (Pl. 8-ból elvehető 1, 2, 4). A végállapot az 1 (nincs kisebb osztója), aki ide kerül, az veszít.

Egy másik pálcikás játék (Osztó-szabály)

Szabályok: Egy halomból k pálcika esetén tetszőleges olyan x számú pálcika elvehető, amelyre igaz, hogy $x < k$ és x osztója k-nak. (Pl. 8-ból elvehető 1, 2, 4). A végállapot az 1 (nincs kisebb osztója), aki ide kerül, az veszít.

Állapotok osztályozása (1 . . . 9):

k	1	2	3	4	5	6	7	8	9
Áll.	V	NY	V	NY	V	NY	V	NY	V

Következtetés:

- Az 1 vesztő pozíció.
- Minden páratlan számból csak páratlan számú pálcikát vehetünk el (mivel páratlan számnak csak páratlan osztói vannak). Páratlanból páratlant kivonva **mindig páros** számot kapunk.
- Minden páros számból elvehetünk 1-et, amivel **páratlan** számot hozunk létre.
- **Szabály:** Minden páros állapot nyerő, minden páratlan állapot vesztő.

A klasszikus Nim játék

A Nim játék központi szerepet játszik a játékelméletben, mivel rengeteg összetettebb játék visszavezethető rá.

Szabályok:

- Adott n darab halom, a halmokban $[x_1, x_2, \ldots, x_n]$ pálcika van.
- A játékosok felváltva lépnek. Egy lépésben ki kell választani **egyetlen** nem üres halmot, és abból tetszőleges számú (legalább 1) pálcikát el kell venni.
- Az nyer, aki az utolsó pálcikát elveszi. A játék vége a $[0, 0, \ldots, 0]$ állapot.

A klasszikus Nim játék

A Nim játék központi szerepet játszik a játékelméletben, mivel rengeteg összetettebb játék visszavezethető rá.

Szabályok:

- Adott n darab halom, a halmokban $[x_1, x_2, \ldots, x_n]$ pálcika van.
- A játékosok felváltva lépnek. Egy lépésben ki kell választani **egyetlen** nem üres halmot, és abból tetszőleges számú (legalább 1) pálcikát el kell venni.
- Az nyer, aki az utolsó pálcikát elveszi. A játék vége a $[0, 0, \ldots, 0]$ állapot.

Elemzés (Bouton-tétel, 1901): Bármely Nim állapot osztályozható a halomméretek **Nim-összegével**, ami a számok bitenkénti kizáró vagy (**XOR**, $\oplus$) kapcsolata:

$$s = x_1 \oplus x_2 \oplus \cdots \oplus x_n$$

- Ha $s = 0 \implies$ a pozíció **vesztő** (V).
- Ha $s \neq 0 \implies$ a pozíció **nyerő** (NY).

Nim játék – Optimális stratégia

Hogyan tartható fenn a nyerő pozíció a Nim-összeg segítségével?

1. Vesztő állapotból ($s = 0$): Bármit lépünk, egy halom mérete megváltozik, így a XOR összeg bitjei elhangolódnak $\Longrightarrow$ a lépésünk után az összeg biztosan $s \neq 0$ (nyerő) lesz az ellenfélnek.

2. Nyerő állapotból ($s \neq 0$): Mindig létezik olyan lépés, amellyel az összeg 0-vá tehető. Ehhez keresni kell egy olyan k-adik halmot, ahol $x_k \oplus s < x_k$. Ha a halom méretét lecsökkentjük erre az értékre, az új Nim-összeg pontosan 0 lesz.

Ilyen halom mindig az lesz, amelynek bináris alakjában az 1-es bit megegyezik az s legbaloldalibb (legnagyobb helyiértékű) 1-es bitjével.

Nim stratégia – Példa

Tekintsük a `[10, 12, 5]` játékállapotot. Határozzuk meg a nyerő lépést!

A Nim-összeg kiszámítása:

10	1010
12	1100
5	0101
3	00$\underline{1}$1 ← Az `s` legbaloldalibb 1-ese a 2-es helyiértéken van.

A 10-es halom bináris alakjában (10$\underline{1}$0) ezen a helyen 1-es áll. A halom új kívánt mérete: $10 \oplus 3 = 9$ (1001).

A lépés: Vegyünk el 1 pálcikát a 10-es halomból. Az új állapot `[9, 12, 5]` lesz, amely már vesztő pozíció az ellenfélnek:

9	1001
12	1100
5	0101
0	0000

A Misère Nim játék

Misère játék: A játék célja az ellentétére fordul: az a játékos **veszít**, aki az utolsó pálcikát kénytelen elvenni.

A Misère Nim játék

Misère játék: A játék célja az ellentétére fordul: az a játékos **veszít**, aki az utolsó pálcikát kénytelen elvenni.

Stratégia: A Misère Nim-et szinte végig pontosan ugyanúgy kell játszani, mint a normál Nim-et (XOR összeg 0-ra hozása), egészen a játék legvégéig.

A kritikus pillanat: Akkor kell stratégiát váltani, amikor a lépésünk után **már az összes halom legfeljebb 1 pálcikát tartalmazna**.

- **Normál játékban** úgy lépnénk, hogy **páros** számú 1 méretű halom maradjon (XOR összeg $= 0$).
- **Misère játékban** úgy kell lépnünk, hogy **páratlan** számú 1 méretű halom maradjon.

Mivel a kritikus állapot előtt mindig pontosan egy darab 1-nél nagyobb halom van, ezt a halmot tetszés szerint lecsökkenthetjük 0 vagy 1 méretűre, így könnyen kikényszeríthetjük a páratlan számú $1 - \mathtt{s}$ halmot.

Kitekintés: a Sprague–Grundy tétel

Megjegyzés: A Sprague-Grundy tétel az IOI Syllabuson kívül esik, de az ismerete szemléletformáló.

A tétel kiterjeszti a Nim játék nyerő stratégiáját **minden** olyan játékra, amely megfelel az alábbi feltételeknek (impartiális játékok):

- Két játékos váltakozva lép, nincsenek véletlen elemek.
- A megengedett lépések csak a pozíciótól függnek, nem attól, ki következik.
- A játék akkor ér véget, ha egy játékos nem tud lépni (ő veszít).
- A játék biztosan véget ér véges számú lépésben.

Alapelv: Minden játékállapothoz hozzárendelünk egy nemnegatív egész számot, az úgynevezett **Grundy-számot**. Ez a szám tökéletesen ekvivalens egy adott méretű Nim-halommal.

A Grundy-szám és a mex függvény

Egy állapot Grundy-száma az abból elérhető közvetlen szomszédos állapotok Grundy-számainak **mex** értéke:

$$g = \mathrm{mex}(\{g_1, g_2, \ldots, g_m\})$$

A **mex** (*minimum excluded*) függvény megadja a halmazban nem szereplő **legkisebb nemnegatív egész** számot.

Példák a mex függvényre:

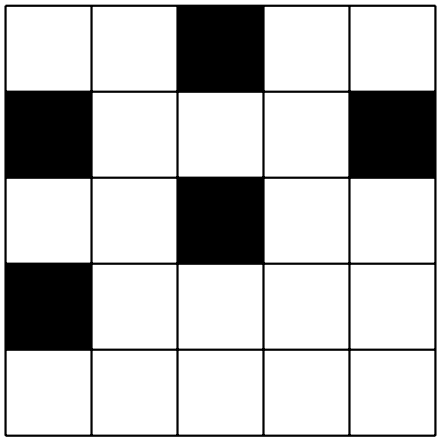
- $\mathrm{mex}(\emptyset) = 0$ (A végállapotok Grundy száma mindig 0)
- $\mathrm{mex}(\{0, 1, 3\}) = 2$
- $\mathrm{mex}(\{1, 2, 4\}) = 0$

Tulajdonságok:

- $g = 0 \implies$ Vesztő pozíció (V). Csak pozitív állapotokba lehet lépni.
- $g > 0 \implies$ Nyerő pozíció (NY). Biztosan lehet lépni 0 Grundy-számú állapotba, sőt minden $0 \ldots g - 1$ közötti értékű állapot elérhető.

Példa – Labirintus játék

Szabályok: Egy rácson (labirintusban) falak és folyosók vannak. Egy bábu áll egy mezőn. Egy lépésben a bábu tetszőleges számú mezőt léphet **balra** vagy **felfelé**, de nem léphet át falon és nem állhat falra. Az veszít, aki nem tud lépni. (A fekete mezők a falak.)



Példa – Labirintus játék

Szabályok: Egy rácson (labirintusban) falak és folyosók vannak. Egy bábu áll egy mezőn. Egy lépésben a bábu tetszőleges számú mezőt léphet **balra** vagy **felfelé**, de nem léphet át falon és nem állhat falra. Az veszít, aki nem tud lépni. (A fekete mezők a falak.)

0	1	■	0	1
■	0	1	2	■
0	2	■	1	0
■	3	0	4	1
0	4	1	3	2

Minden szabad mező egy-egy játékállapot. Határozzuk meg a Grundy-számokat felülről lefelé, balról jobbra!

A jobb alsó sarok értéke 2, mivel innen léphetünk balra a 3, 1, 4, 0 értékekre, vagy fel az 1, 0 értékekre, és a legkisebb hiányzó érték belőlük a 2. Így ez nyerő pozíció, kezdésként az egyik 0-ra lépünk (felfelé 2-t lépve vagy balra 4-et lépve).

Összetett játékok – Részjátékok

Mi történik, ha a játék több független részjátékból áll? (Pl. egyszerre 3 külön labirintusban van 1-1 bábu, és a soron következő játékos kiválaszt **egyet**, majd abban lép).

A játék akkor ér véget, ha már egyik részjátékban sem lehet lépni.

A Sprague-Grundy tétel fő eredménye: Az összetett játék teljes Grundy-száma a független részjátékok egyedi Grundy-számainak **Nim-összege** (XOR összege):

$$G = g_1 \oplus g_2 \oplus \cdots \oplus g_k$$

Ezáltal az összetett játékot pontosan úgy játszhatjuk, mintha egy klasszikus Nim játékot játszanánk, ahol a halomméretek a részjátékok pillanatnyi Grundy-számai.

Játék felosztása részjátékokra

Néha egy lépés hatására egyetlen játékállapot önmagában **kettéágazik** független részjátékokra.

A kombinált állapot szabálya: Ha egy szabályos lépéssel az a és b Grundy-számú független részjátékok keletkeznek, akkor az így kapott összetett pozíció Grundy-száma a részek Nim-összege, vagyis: $f(a) \oplus f(b)$ lesz.

A mex függvény kiterjesztése: Ebből kifolyólag egy n állapotú, kettéágazó játék Grundy-számát úgy kapjuk meg, hogy az összes elérhető állapotpár kombinált értékére alkalmazzuk a minimum-kizárás függvényt:

$$f(n) = \text{mex}(\{f(a) \oplus f(b) \mid \text{minden szabályos } n \to (a, b) \text{ lépésre}\})$$

A stratégia: A játékot ugyanúgy játsszuk, mint a klasszikus Nim-et. A célunk mindig olyan lépést választani, amely után a keletkező részjátékok Grundy-számainak XOR összege pontosan 0 lesz, mert ez a vesztő pozíciót kényszeríti rá az ellenfélre.

Példa az osztódó játékra: Grundy játéka

Szabályok: Kezdetben egy `n` pálcikás halom van. Egy lépésben egy tetszőleges halmot két **nem üres és különböző méretű** halomra kell osztani. Az utolsó lépő nyer.

Példa az osztódó játékra: Grundy játéka

Szabályok: Kezdetben egy n pálcikás halom van. Egy lépésben egy tetszőleges halmot két **nem üres és különböző méretű** halomra kell osztani. Az utolsó lépő nyer.

A Grundy-számok $n = 8$-ra ($f(n)$):

- $f(1) = 0$ és $f(2) = 0$, mivel ezeket nem lehet felosztani.
- $f(3) = \text{mex}(\{f(1) \oplus f(2)\}) = \text{mex}(\{0 \oplus 0\}) = \text{mex}(\{0\}) = 1$
- $f(4) = \text{mex}(\{f(1) \oplus f(3)\}) = \text{mex}(\{0 \oplus 1\}) = \text{mex}(\{1\}) = 0$ *(A $2 + 2$ nem szabályos!)*
- $f(5) = \text{mex}(\{f(1) \oplus f(4),\ f(2) \oplus f(3)\}) = \text{mex}(\{0 \oplus 0,\ 0 \oplus 1\}) = \text{mex}(\{0, 1\}) = 2$
- $f(6) = \text{mex}(\{f(1) \oplus f(5),\ f(2) \oplus f(4)\}) = \text{mex}(\{0 \oplus 2,\ 0 \oplus 0\}) = \text{mex}(\{2, 0\}) = 1$
- $f(7) = \text{mex}(\{f(1) \oplus f(6),\ f(2) \oplus f(5),\ f(3) \oplus f(4)\}) = \text{mex}(\{0 \oplus 1,\ 0 \oplus 2,\ 1 \oplus 0\}) = 0$
- $f(8) = \text{mex}(\{f(1) \oplus f(7),\ f(2) \oplus f(6),\ f(3) \oplus f(5)\}) = \text{mex}(\{0 \oplus 0,\ 0 \oplus 1,\ 1 \oplus 2\}) = \text{mex}(\{0, 1, 3\}) = 2$

Példa az osztódó játékra: Grundy játéka

Szabályok: Kezdetben egy n pálcikás halom van. Egy lépésben egy tetszőleges halmot két **nem üres és különböző méretű** halomra kell osztani. Az utolsó lépő nyer.

A Grundy-számok $n = 8$-ra ($f(n)$):

- $f(1) = 0$ és $f(2) = 0$, mivel ezeket nem lehet felosztani.
- $f(3) = \text{mex}(\{f(1) \oplus f(2)\}) = \text{mex}(\{0 \oplus 0\}) = \text{mex}(\{0\}) = 1$
- $f(4) = \text{mex}(\{f(1) \oplus f(3)\}) = \text{mex}(\{0 \oplus 1\}) = \text{mex}(\{1\}) = 0$ *(A $2 + 2$ nem szabályos!)*
- $f(5) = \text{mex}(\{f(1) \oplus f(4),\ f(2) \oplus f(3)\}) = \text{mex}(\{0 \oplus 0,\ 0 \oplus 1\}) = \text{mex}(\{0, 1\}) = 2$
- $f(6) = \text{mex}(\{f(1) \oplus f(5),\ f(2) \oplus f(4)\}) = \text{mex}(\{0 \oplus 2,\ 0 \oplus 0\}) = \text{mex}(\{2, 0\}) = 1$
- $f(7) = \text{mex}(\{f(1) \oplus f(6),\ f(2) \oplus f(5),\ f(3) \oplus f(4)\}) = \text{mex}(\{0 \oplus 1,\ 0 \oplus 2,\ 1 \oplus 0\}) = 0$
- $f(8) = \text{mex}(\{f(1) \oplus f(7),\ f(2) \oplus f(6),\ f(3) \oplus f(5)\}) = \text{mex}(\{0 \oplus 0,\ 0 \oplus 1,\ 1 \oplus 2\}) = \text{mex}(\{0, 1, 3\}) = 2$

Stratégia $n = 8$ esetén: Mivel $f(8) = 2 \neq 0$, ez egy nyerő pozíció. Úgy kell lépnünk, hogy a részjátékok XOR összege 0 legyen. A fentiek alapján a nyerő lépés az $1 + 7$ **halmok kialakítása**, hiszen $f(1) \oplus f(7) = 0 \oplus 0 = 0$.

Összegzés – Megoldási stratégia

A kombinatorikus játékok megoldásakor az állapottér mérete határozza meg a módszerünket:

1. **Kis állapottér esetén (Dinamikus programozás):**
   - Ha az állapotok száma nem túl nagy, **dinamikus programozással** (rekurzió + memorizálás) hátulról visszafelé haladva szisztematikusan kiszámolható az összes nyerő (`NY`) és vesztő (`V`) pozíció.

2. **Hatalmas állapottér esetén (Konstruktív/Invariáns módszer):**
   - Bizonyos játékoknál fel lehet fedezni a vesztő mezők egy **közös megmaradó (invariáns) tulajdonságát**.
   - *Példák:* Pálcikás játéknál az oszthatóság, Nim játéknál a XOR összeg.
   - **A nyerő stratégia mechanizmusa:** Olyan matematikai vagy logikai tulajdonságot kell találnunk, amelyre igaz, hogy:
     1. A „jó” (vesztő) állapotból az ellenfél **bármely szabályos lépéssel elrontja** a mintát (átviszi egy nyerő állapotba).
     2. Mi a „rossz” (nyerő) állapotból **mindig vissza tudjuk állítani** a kívánt tulajdonságot, és visszaadjuk a vesztő pozíciót az ellenfélnek.
   - Ez a megközelítés **konstruktív megoldást** ad, így akkor is biztos győzelmet garantál, ha a játékállapotok száma egyébként csillagászati lenne.

+1 játék: Csúcsbejárás gráfon

Szabályok: Adott egy összefüggő, irányítatlan `G = (V, E)` gráf. Alíz és Bob felváltva lépnek egy bábura a gráf élein keresztül.

- A kezdőcsúcsot szabadon **Alíz választja meg** a játék legelején.
- A játék során **nem szabad olyan csúcsra lépni**, amelyet már korábban érintett a bábu (minden csúcs legfeljebb egyszer látogatható meg).
- Az a játékos veszít, aki a soron következő lépésben már nem tud szabályos mezőre lépni.

+1 játék: Csúcsbejárás gráfon

Szabályok: Adott egy összefüggő, irányítatlan $G = (V, E)$ gráf. Alíz és Bob felváltva lépnek egy bábura a gráf élein keresztül.

- A kezdőcsúcsot szabadon **Alíz választja meg** a játék legelején.
- A játék során **nem szabad olyan csúcsra lépni**, amelyet már korábban érintett a bábu (minden csúcs legfeljebb egyszer látogatható meg).
- Az a játékos veszít, aki a soron következő lépésben már nem tud szabályos mezőre lépni.

Megoldás: A játék kimenetele a gráf **maximális párosításától** függ:

- Ha a gráfban **van teljes párosítás** $\Longrightarrow$ **Bob nyer**.
- Ha a gráfban **nincs teljes párosítás** $\Longrightarrow$ **Alíz nyer**.

Hogy lépjenek a játékosok? Mi legyen Alíz stratégiája?

A csúcsbejárós játék stratégiája és invariánsa

Hogyan garantálható a győzelem?

- **Ha van teljes párosítás (Bob stratégiája):** Alíz bárhova rakja a bábut, az a csúcs le van fedve a párosítás által. Bob lépése mindig az Alíz által választott csúcs **párosításbeli párja** lesz.
- **Ha nincs teljes párosítás (Alíz stratégiája):** Alíz rögzít egy **maximális párosítást**. A kezdőcsúcsnak egy olyan csúcsot választ, amely **kimaradt** ebből a párosításból (mivel nem teljes, biztosan van ilyen). Ezután amikor Bob lép egyet, Alíz mindig Bob csúcsának a **párosításbeli párjára** lép.

Miért működik ez?

A csúcsbejárós játék stratégiája és invariánsa

Hogyan garantálható a győzelem?

- **Ha van teljes párosítás (Bob stratégiája):** Alíz bárhova rakja a bábut, az a csúcs le van fedve a párosítás által. Bob lépése mindig az Alíz által választott csúcs **párosításbeli párja** lesz.
- **Ha nincs teljes párosítás (Alíz stratégiája):** Alíz rögzít egy **maximális párosítást**. A kezdőcsúcsnak egy olyan csúcsot választ, amely **kimaradt** ebből a párosításból (mivel nem teljes, biztosan van ilyen). Ezután amikor Bob lép egyet, Alíz mindig Bob csúcsának a **párosításbeli párjára** lép.

Miért működik ez?

- A játék **invariánsa**: A nyerő játékos mindig egy **párosításbeli élen** lép, a vesztő játékos pedig kénytelen egy **párosításon kívüli élen** lépni.
- Ha a vesztő játékos tudna utoljára lépni, akkor a bejárt útvonal egy **javítóút** lenne. E mentén a párosítást megfordítva egy nagyobb párosítást kapnánk, ami ellentmond annak, hogy maximális párosítást vettünk.

Algoritmikus háttér:

- **Páros gráf esetén:** A maximális párosítás a *magyar módszerrel* megtalálható.
- **Általános gráf esetén:** A feladat a *Edmonds-féle Blossom algoritmussal* oldható meg, azonban ez **az IOI Syllabuson kívüli**.

Pontgyűjtő játékok és a Minimax

Általános megoldási stratégiák, kiértékelési fák és az értékek maximalizálása

Általános stratégia: A Minimax algoritmus

Amikor a játék nem csupán nyerő/vesztő kimenetelű (bináris), hanem a cél a **pontszám maximalizálása**, a pozíciókat számértékekkel látjuk el.

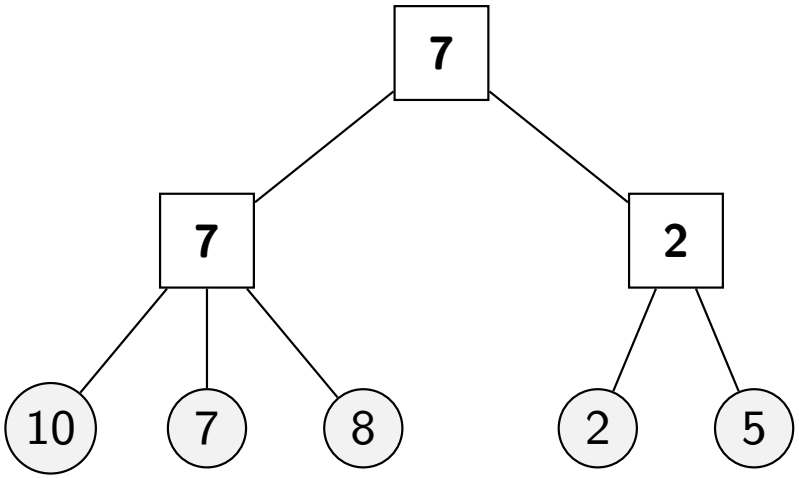
A két játékos szerepe:

- **MAX játékos (Mi):** Olyan lépést választ, amely a számára elérhető pontszámot **maximalizálja**.
- **MIN játékos (Ellenfél):** Feltételezzük, hogy optimálisan játszik, tehát ő a mi pontszámunkat próbálja **minimalizálni** (ezzel a sajátját maximalizálva).

A döntési mechanizmus: Minden állapot értéke a belőle elérhető állapotok értékeitől függ. A játékot egy **játékfával** modellezzük, ahol a levelek a játék végeztével kapott konkrét pontszámok.

Minimax kiértékelési fa

A fa gyökere a jelenlegi állapot. A szinteken felváltva a MAX és MIN játékos dönt. Az értékek alulról (a levelektől) felfelé terjednek.



← **MAX szint** (mi: `max(7, 2)` = 7)

← **MIN szint** (ő: `min(10, 7, 8)` = 7)

← **Levelek** (végállapotok értékei)

Szabály:

- Ha **mi** jövünk: `V = max(gyerekek értékei)`
- Ha az **ellenfél** jön: `V = min(gyerekek értékei)`

Minimax megvalósítása: Rekurzió és DP

A játékfa bejárása mélységi kereséssel (DFS) történik. Mivel az állapottér gyakran hatalmas, két fontos optimalizálást alkalmazhatunk:

1. Memorizáció / Dinamikus programozás: Ha ugyanaz az állapot (pl. ugyanaz a pálcikaszám vagy táblaállás) többször is előfordul különböző útvonalakon, az értékét csak egyszer számoljuk ki és eltároljuk.

2. Alfa-Béta vágás (Kitekintés): Ha egy ágról már látszik, hogy biztosan nem fogja befolyásolni a végső döntést (mert a korábban talált lehetőségeknél rosszabb a MIN vagy MAX számára), akkor azt az ágat **lemetsszük**, nem járjuk be.

Komplexitás:

- Nyers rekurzió: $O(b^d)$, ahol b az elágazási tényező, d a mélység.
- DP esetén: Az egyedi állapotok száma határozza meg a futásidőt.

Példák

Interaktív IOI feladatok

Számjáték (IOI 1996)

Adott egy pozitív egész számokból álló $a_1, a_2, \ldots, a_n$ sorozat az asztalon. Két játékos felváltva lép, a játékot az Első Játékos kezdi. Egy lépésben a soron lévő játékos elveheti a sorozat **bal vagy jobb szélén** lévő számot. A kiválasztott szám törlődik a tábláról, és hozzáadódik a játékos pontszámához.

A játék vége és a cél: A játék akkor ér véget, ha az összes szám elfogyott. Az Első Játékos nyer, ha az általa gyűjtött számok összege legalább annyi, mint a Második Játékosé. A Második Játékos (a gép) szintén optimálisan játszik.

Garantált feltétel: Ha a számok kezdeti száma (`n`) **páros**, az Első Játékosnak mindig van egy egyszerű, matematikai nyerő stratégiája.

Kommunikációs interfész (`Play` modul):

- `StartGame()`: Elindítja a játékot.
- `MyMove(lep)`: Lépés küldése. Értéke `'L'` (bal) vagy `'R'` (jobb).
- `YourMove()`: Lekéri a gép azonnali válaszlépését (`'L'` vagy `'R'`).

Paritásos stratégia (Az alapprobléma megoldása)

Bármelyik játékállás leírható egy (i, j) indexpárral, ami azt jelenti, hogy a táblán éppen az $a_i, \ldots, a_j$ részsorozat található. Mivel n páros, az Első Játékos rákényszerítheti az ellenfelet egy fix mintára.

A trükk: Vegyük a kezdeti sorozat elemeinek indexeit:

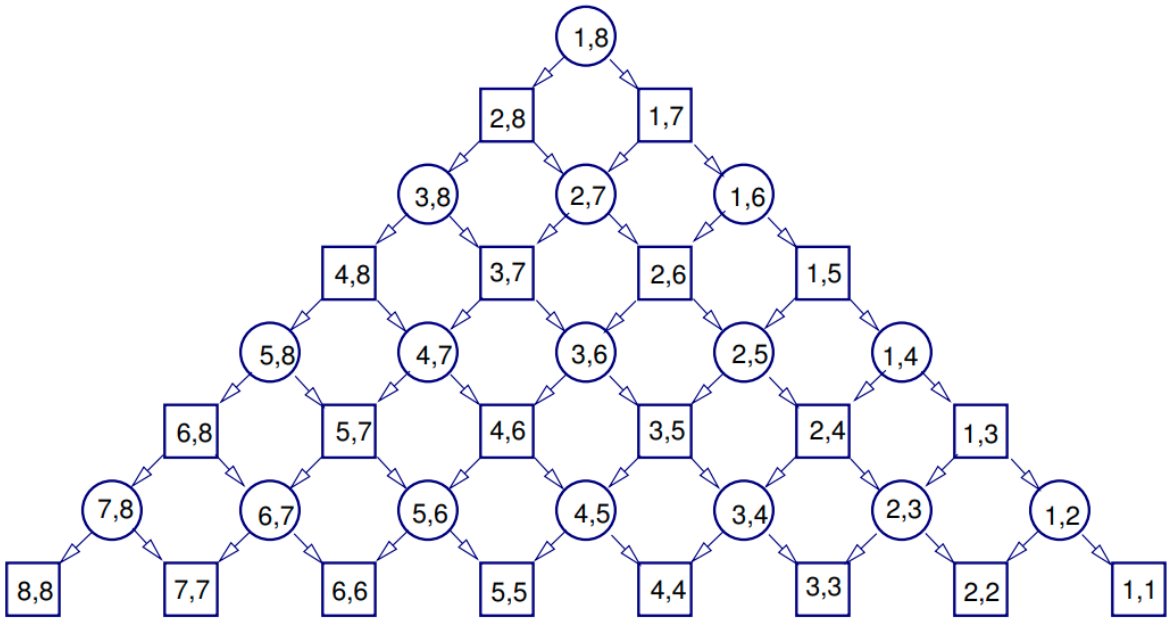
- **Páratlan pozíciók összege:** $S_{ptlan} = a_1 + a_3 + a_5 + \ldots$
- **Páros pozíciók összege:** $S_{paros} = a_2 + a_4 + a_6 + \ldots$

Végrehajtás: Ha $S_{ptlan} \geq S_{paros}$, az Első Játékos elveszi a_1-et (páratlan). Ezzel a Második Játékos előtt két páros indexű elem (a_2 és a_n) marad. Bármelyiket is választja, az Első Játékos a következő körben újra egy páratlan indexű elemet tud elvenni.

Így az Első Játékos garantáltan megszerzi az összes páratlan (vagy fordított esetben az összes páros) pozíción lévő számot, ezzel biztosítva a győzelmet.

Pontszám maximalizálás

Ha a célunk nemcsak a győzelem, hanem a **maximálisan elérhető pontszám** megszerzése (optimális ellenfél mellett), a játékot egy irányított aciklikus gráffal (DAG) modellezhetjük.

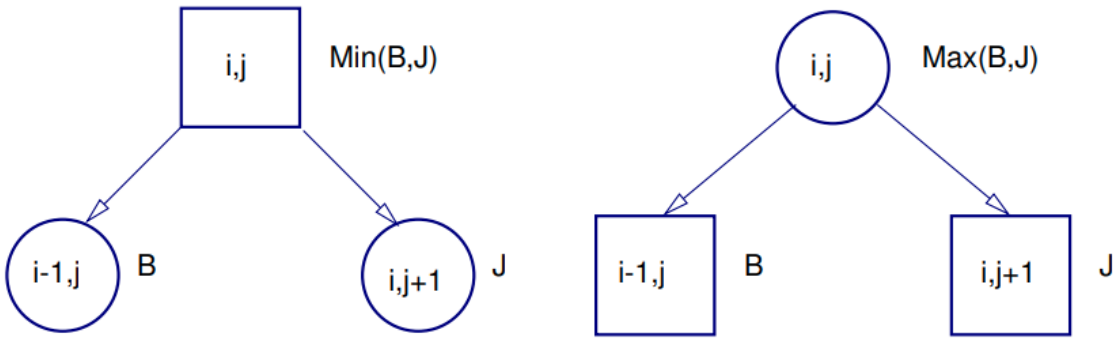


A játékállások gráfja `n` = 8 esetén. Körrel jelölt állásból (`i` + `j` páratlan) az Első, négyzettel jelölt állásból (`i` + `j` páros) a Második játékos lép.

A Minimax szabály és dinamikus programozás

Jelölje `dp(i, j)` azt a maximális pontszámot, amit az Első Játékos garantáltan meg tud szerezni az `(i, j)` játékállásból indulva. Ez az érték az alábbi rekurzív minimax képlettel határozható meg:

$$\texttt{dp(i, j)} = \begin{cases} 0, & \text{ha } \texttt{i} = \texttt{j} \\ \max(a_i + \texttt{dp(i+1, j)},\ a_j + \texttt{dp(i, j-1)}), & \text{ha } \texttt{i} < \texttt{j} \text{ és } \texttt{i} + \texttt{j} \text{ pár} \\ \min(\texttt{dp(i+1, j)},\ \texttt{dp(i, j-1)}), & \text{ha } \texttt{i} < \texttt{j} \text{ és } \texttt{i} + \texttt{j} \text{ pár} \end{cases}$$



A döntési logika működése:

- **Maximálás (Első Játékos):** Amikor mi jövünk, úgy választunk oldalt, hogy az elvett szám (a_i vagy a_j) és a belőle következő állapot optimális értékének összege a lehető legnagyobb legyen.
- **Minimálás (Második Játékos):** Amikor a gép jön, ő is a saját pontjait maximalizálja. Mivel a gép pontjai a mi veszteségünk, ez számunkra azt jelenti, hogy a gép a számunkra elérhető `dp` értéket fogja **minimalizálni**.

Algoritmus és lépéstárolás

Mivel a korlát $2 \leq \texttt{n} \leq 1000$, egy 1000×1000-es táblázatban előre ki tudjuk számolni az összes `dp(i, j)` értéket a játék megkezdése előtt (dinamikus programozás alulról felfelé építkezve, az intervallum hossza szerint növekvő sorrendben).

Az optimális lépések elmentése: A kiszámítás során egy kiegészítő `EnLep[i][j]` mátrixban rögzítjük az optimális döntést minden olyan állapothoz, ahol mi jövünk:

- `'L'` (Left), ha $\texttt{a}_\texttt{i} + \texttt{dp(i + 1, j)} > \texttt{a}_\texttt{j} + \texttt{dp(i, j - 1)}$
- `'R'` (Right) egyébként.

A játék menete: A program a meccs során nem számol újra: amikor az `(i, j)` állásban az Első Játékos következik, egyszerűen kiolvassa az `EnLep[i][j]` értékét, elküldi a `MyMove` függvénnyel, majd a kapott ellenfél-lépés alapján frissíti az `i` vagy `j` mutatókat a következő saját körére.

Ioiwari Game (IOI 2001)

Az Ioiwari egy klasszikus Mancala-típusú játék, amelyet két játékos játszik egy kör alakú táblán, 7 mélyedéssel (pit) és 2 bankkal (minden játékosnak saját). A játék kezdetén 20 gyöngyöt osztanak szét véletlenszerűen úgy, hogy minden mélyedésben $2 \leq$ gyöngy ≤ 4 legyen. A játékosok felváltva lépnek.

A lépés mechanizmusa: A soron lévő játékos kiürít egy nem üres mélyedést, és a gyöngyöket óramutató járásával megegyező irányban egyesével elosztja:

- **Ha több mint 1 gyöngy van a kézben:** Ha az aktuális mélyedésben már 5 gyöngy van, egyet kiveszünk és a **saját bankba** tesszük. Különben lerakunk egyet a mélyedébe.
- **Ha pontosan 1 gyöngy maradt a kézben:** Ha az aktuális mélyedésben 1 és 4 közötti gyöngy van, az összes ottani gyöngyöt a kézben lévővel együtt a **saját bankba** seperjük. Ha a mélyedésben 0 vagy 5 gyöngy van, az utolsó gyöngyöt az **ellenfél bankjába** kell tennünk.

Végfeltétel: A játéknak akkor van vége, ha egy lépés után az összes mélyedés kiürül. A győztes az, akinek a legtöbb gyöngy van a bankjában.

Interfész és elvárások

A kezdő játékosnak **mindig van nyerő stratégiája**. Olyan programot kell írnunk, amely első játékosként indulva garantáltan nyer az optimálisan játszó értékelő ellenféllel megmérkőzve.

Kommunikációs protokoll:

- A program elindulásakor beolvassa a kezdőállapotot a standard inputról (7 darab egész szám, pl. `4 3 2 4 2 3 2`).
- Ha a mi programunk jön: kiírjuk a választott mélyedés indexét (1-től 7-ig) a standard outputra (és ürítjük a puffert: `fflush(stdout)`).
- Ha az ellenfél jön: beolvassuk az ellenfél lépését (mélyedés indexe) a standard inputról.

Pontozás (Meccsenként):

- **4 pont:** Ha a programunk megnyeri a játszmát.
- **2 pont:** Döntetlen esetén.
- **0 pont:** Ha a program kikap. Az ellenfél optimális, így egyetlen hiba is azonnali vereséghez vezethet!

A játék gráf modellje

Modellezzük a játékot egy irányított gráffal, ahol a csúcsok az `(w, B)` állapotok:

- `w` $\in \{1, 2\}$: a soron következő játékos azonosítója.
- `B`: a tábla aktuális állapota (a mélyedésekben lévő gyöngyök száma).

Kulcstulajdonság: DAG (körmentes gráf) Mivel minden egyes lépés során legalább 1 gyöngy véglegesen bekerül valamelyik bankba, a mélyedésekben lévő gyöngyök száma **szigorúan monoton csökken**. A játék gráfja így körmentes (DAG), ami lehetővé teszi a dinamikus programozás / rekurzió memorizálással alkalmazását.

Kombinatorikai méret: Mivel egyetlen mélyedésben sem lehet 5-nél több gyöngy a lépések során, minden mélyedés állapota 0 és 5 között mozoghat (6 lehetőség). Egy állapothoz tartozó egyedi azonosító (ID) felírható egy 7-jegyű, 6-os számrendszerbeli számként:

$$\text{MaxN} = 6^7 = 279\,936$$

Minimax és dinamikus programozás

Definiáljuk a `Diff(w, B)` függvényt, amely az Első Játékos által elérhető **legjobb pontkülönbséget** ($\text{Bank}_1 - \text{Bank}_2$) jelöli a `B` állapotból, feltételezve, hogy a Második Játékos is optimálisan játszik.

Rekurzív reláció (Minimax):

$$\texttt{Diff(w, B)} = \begin{cases} 0, & \text{ha } \texttt{B} \text{ teljesen üres} \\ \max_i\{\texttt{Diff(2, Move(B, i))} + \texttt{D(B, i)}\}, & \text{ha } \texttt{w} = 1 \\ \min_i\{\texttt{Diff(1, Move(B, i))} + \texttt{D(B, i)}\}, & \text{ha } \texttt{w} = 2 \end{cases}$$

Ahol `Move(B, i)` a lépés utáni tábla, `D(B, i)` pedig az adott lépésben a bankokba kerülő gyöngyök különbsége ($\Delta\text{Bank}_1 - \Delta\text{Bank}_2$).

Nyerő stratégia feltétele: Az Első Játékosnak pontosan akkor van nyerő stratégiája a kezdő $\texttt{B}_0$ állapotból, ha $\texttt{Diff(1, B}_0\texttt{)} > 0$.

Memorizáció és optimalizáció

A tiszta rekurzió a felesleges újraszámítások miatt túl lassú lenne, de a memoizációs technika segítségével a futásidő drasztikusan lecsökkenthető.

Megvalósítás részletei:

- Létrehozunk egy állapot-táblázatot, amely a kiszámított `Diff(w, B)` értékeket és az optimális lépést tárolja.
- **Tárigény:** Az állapottér $2 \times 6^7 \approx 560\,000$ bejegyzés. Ez `char` vagy `short` típusokkal mindössze pár megabájt memóriát igényel, ami kényelmesen belefér a korlátokba.
- **Forgatási szimmetria (Opcionális):** Mivel a tábla kör alakú, a forgatással ekvivalens állapotok azonosak. Ha minden állapothoz a forgatási ekvivalensei közül a lexikografikusan legkisebb base-6 értéket rendeljük, a memóriaszükséglet a **hetedére** (1/7) csökkenthető.
- **Időkomplexitás:** Arányos a gráf éleinek számával. Mivel minden állapotból legfeljebb 7 lehetséges lépés (él) indul, a műveletigény legfeljebb $7 \times \text{MaxN} \approx 1.9 \times 10^6$ lépés, ami belefér az időlimitbe.

Feladatok

Önálló munkára

Két játékos egy bábuval

https://njudge.hu/problemset/main/M113_4c/

Adott egy $N \times N$-es négyzethálós tábla, amelynek minden mezőjén egy-egy természetes szám található. A játékot két játékos játssza, akik felváltva mozgatnak egyetlen **közös bábut**.

A játék szabályai:

- A bábu kezdetben a tábla bal felső sarkában, a $(0, 0)$ koordinátán áll.
- Egy lépésben a bábuval csak a szomszédos mezőre lehet lépni, kizárólag **lefelé** vagy **jobbra**.
- A játékot az Első Játékos kezdi.
- Az Első Játékos megkapja a pontot minden olyan mező után, **amelyre ő lépett**.

A játék vége és a célok: A játék akkor ér véget, amikor a bábu eléri a tábla jobb alsó, $(N-1, N-1)$ koordinátájú mezőjét.

- **Első Játékos:** Arra törekszik, hogy a saját lépéseivel a lehető **legtöbb összpontot** szerezze meg.
- **Második Játékos:** Arra törekszik, hogy a saját döntéseivel **minimalizálja** az Első Játékos által megszerezhető pontokat (akadályozza őt).

A feladat: Írj programot, amely az Első Játékos szerepét játszva optimális stratégiát valósít meg a Második Játékos ellenében!

XOR Break — Game Version (Codeforces 1934D2)

https://codeforces.com/contest/1934/problem/D2

Alice és Bob egy interaktív játékot játszanak egy pozitív n egésszel ($1 \le n \le 10^{18}$). A játékosok felváltva lépnek, és a célunk Alice-ként megnyerni a játékot. Szabadon megválaszthatjuk, hogy **elsőként** (`first`) vagy **másodikként** (`second`) szeretnénk-e kezdeni.

A játékmenet egy körben:

1. A soron lévő játékos vesz egy p számot, és **szétbontja** két kisebb pozitív egészre (p_1, p_2), amelyekre igaz:

$$0 < p_1 < p, \quad 0 < p_2 < p \quad \text{és} \quad p_1 \oplus p_2 = p$$

2. Ha a játékos nem tud ilyen p_1, p_2 párost mutatni, **azonnal veszít**.
3. Ha a bontás sikeres, az **ellenfél választja ki**, hogy a p_1 vagy a p_2 számmal folytatódjon-e a játék. A másik szám kiesik.

Az interakció protokollja (tesztesetenként):

- Beolvassuk az n számot, majd kiírjuk: `"first"` vagy `"second"`.
- A mi körünkben kiírunk két számot (p_1 p_2), majd ürítjük a puffert (`flush`).
- Az ellenfél körében beolvasunk két számot. Ha a beolvasott érték `0 0`, akkor az ellenfél nem tudott lépni, így **megnyertük** az adott tesztesetet.

Korlátok: Legfeljebb 63 saját bontási lépés engedélyezett tesztesetenként.

Rectangle Game (IOI 2005)

A játék szabályai: Adott egy `X` × `Y` méretű téglalap (`X`, `Y` pozitív egészek). A két játékos felváltva vág.

- Egy lépésben a téglalapot egyetlen egyenes vágással (vízszintesen vagy függőlegesen) két kisebb téglalapra osztjuk.
- A vágás után a **kisebb területű darabot eldobjuk**, a nagyobb darabot pedig átadjuk a másik játékosnak. (Ha pontosan felezzük, az egyik felet eldobjuk.)
- Az a játékos, aki egy 1 × 1-es téglalapot kap, már nem tud szabályosan lépni, így **elveszíti a játékot**.

Az interaktív környezet: Nem ismerjük előre az ellenfél lépéseit. Egy speciális könyvtáron keresztül kommunikálunk:

- `dimension_x()`, `dimension_y()`: Lekéri a jelenlegi méreteket.
- `cut(irány, pozíció)`: Végrehajtja a vágásunkat, majd automatikusan átadja a kört az ellenfélnek, és csak az ellenfél válaszlépése után adja vissza a vezérlést.

Beadható itt (stdin/stdout interaktív környezettel): `https://ioi.contest.codeforces.com/group/32KGsXgiKA/contest/103747/problem/E`

Megoldás

Rectangle Game

Az 50 pontos megoldás: Klasszikus DP

A tesztesetek 50%-ában a kiinduló méretek nem haladják meg a 25-öt ($\texttt{X}, \texttt{Y} \leq 25$). Mivel a játéktér kicsi és teljesen ismert, alkalmazhatjuk a korábban tanultakat:

Állapotok osztályozása (Nyerő vs. Vesztő):

- Egy `(x, y)` állapot **Vesztő (V)**, ha az 1×1-es alapállapot, VAGY ha belőle **minden** szabályos vágással csak Nyerő állapotba tudunk lépni.
- Egy `(x, y)` állapot **Nyerő (NY)**, ha **létezik legalább egy** olyan szabályos vágás, amellyel az ellenfélnek egy Vesztő állapotot adunk át.

Dinamikus Programozás $\texttt{O(X} \cdot \texttt{Y} \cdot \texttt{(X + Y))}$ **időben:** Mivel a játék determinisztikus, egy `nyerő[26][26]` logikai táblázatban alulról felfelé haladva minden állapotra kiszámolhatjuk a státuszt. Ha az aktuális állapot nyerő, a táblázat alapján azonnal látszik, melyik vágás vezet az ellenfél vesztéhez.

A 100 pontos ötlet: Mintafelismerés (Lemma)

Mi a teendő, ha a méretek hatalmasak ($X, Y \leq 10^8$)? A táblázat kitöltése lehetetlen. Ha kiszámoljuk a vesztő helyzeteket kis méretekre, egy nagyon tiszta matematikai mintázat rajzolódik ki:

y \ x	1	2	3	4	5	6	7	8	9	10
1	**V**	.	**V**	.	.	.	**V**	.	.	.
2	.	**V**	.	.	**V**	.	.	.	.	**V**
3	**V**	.	**V**	.	.	.	**V**	.	.	.
4	.	.	.	**V**	.	.	.	.	**V**	.

A Rectangle-játék Lemmája: Egy $n \times m$ méretű téglalap akkor és csak akkor **vesztő pozíció**, ha a megnövelt oldalhosszak hányadosa pontosan kettő valamelyik egész kitevős hatványa:

$$\frac{m+1}{n+1} = 2^k \quad \text{(valamilyen } k \in \mathbb{Z} \text{ esetén)}$$

Példa: Az 1×1 esetén $\frac{1+1}{1+1} = 1 = 2^0$, tehát vesztő. A 3×1 esetén $\frac{1+1}{3+1} = \frac{2}{4} = 2^{-1}$, szintén vesztő.

Nyerő stratégia generálása

A feladat garantálja, hogy a kiinduló állapotból van nyerő stratégia (tehát nem a fenti vesztő arányból indulunk). A célunk olyan vágást eszközölni, amellyel az ellenfelet **vesztő helyzetbe kényszerítjük**.

Tegyük fel, hogy $n > m$ (a szélesség nagyobb, mint a magasság). Olyan új l szélességet szeretnénk kapni a vágásunk után, amelyre teljesül a lemma. Mivel a kisebb darabot eldobjuk, a megmaradó rész hossza $\frac{n}{2} \leq l < n$ között kell legyen.

A pontos célméret (l) kiszámítása a logaritmus segítségével lehetséges:

$$l = 2^{\left\lfloor \log_2 \frac{n+1}{m+1} \right\rfloor} \cdot (m+1) - 1$$

A matematikai bizonyítás garantálja, hogy ez az l érték mindig a szabályos $[\lceil n/2 \rceil, n-1]$ vágási tartományba fog esni. Ha ezt a méretet hagyjuk meg, az ellenfél tehetetlen lesz!